

Performance Evaluation of Event-Driven Banking Microservices Using Apache Kafka and Spring Boot Under High-Concurrency Workloads

Ayshi Shannidhya Panda Department of Computer Science and Engineering
Silicon Institute of Technology
Sambalpur, Odisha, India
asp45624@gmail.com

Abstract—Microservice architectures have become the standard for building scalable financial applications, yet the choice of inter-service communication paradigm—synchronous REST, message-oriented middleware (RabbitMQ), or distributed event streaming (Apache Kafka)—remains insufficiently characterized under realistic banking workloads. This paper presents a systematic empirical evaluation of three communication strategies implemented within *Neptune Bank*, a cloud-native banking platform comprising eight microservices built with Spring Boot 3.5 and Java 21, deployed via Docker on PostgreSQL databases. We design and execute controlled experiments using Apache JMeter and Gatling under graduated concurrency levels (100 to 20,000 concurrent users), measuring throughput, end-to-end latency (p50, p95, p99), CPU and memory utilization, Kafka consumer lag, error rate, and mean time to recovery (MTTR). A communication strategy abstraction layer enables switching between REST, RabbitMQ, and Kafka via Spring Profiles without modifying business logic. We apply Little’s Law, Amdahl’s Law, and queueing-theoretic models to interpret the observed scaling behavior. Fault tolerance experiments simulate broker crashes, service failures, and database slowdowns using chaos engineering techniques. Statistical analysis using ANOVA, confidence intervals, and hypothesis testing validates the significance of observed differences. Results demonstrate that Kafka-based event-driven communication achieves [TO BE FILLED AFTER EXPERIMENTS] improvement in throughput and [TO BE FILLED] reduction in p99 latency compared to synchronous REST under high-concurrency workloads, while RabbitMQ occupies an intermediate position. We identify the concurrency crossover point at which event-driven architecture outperforms synchronous communication and discuss the architectural trade-offs between consistency, latency, throughput, operational complexity, and fault tolerance. All source code, benchmarking scripts, Docker configurations, and raw experimental data are publicly available for reproducibility.

Index Terms—Microservices, Apache Kafka, Spring Boot, Event-Driven Architecture, Performance Evaluation, Distributed Systems, Banking Systems, Message Queuing, RabbitMQ, High Concurrency, Cloud-Native Architecture, Fault Tolerance

I. INTRODUCTION

THE financial services industry has undergone a fundamental architectural transformation over the past decade, migrating from monolithic enterprise applications toward microservice-based architectures that promise improved scalability, independent deployability, and technology heterogeneity [1], [2]. Modern banking platforms must simultaneously satisfy stringent requirements: sub-second transaction pro-

cessing, 99.99% availability, regulatory compliance for audit trails, and the ability to handle traffic spikes that can exceed $10\times$ normal volumes during events such as month-end salary deposits, festive-season sales, or flash promotions [3].

A critical design decision in any microservice architecture is the choice of inter-service communication paradigm. Synchronous communication via REST APIs provides simplicity, immediate consistency, and straightforward debugging, but introduces tight temporal coupling between services—if a downstream service is slow or unavailable, the upstream caller blocks or fails [4]. Asynchronous messaging through brokers such as RabbitMQ provides temporal decoupling with reliable message delivery guarantees, but introduces operational complexity and eventual consistency semantics [5]. Distributed event streaming platforms such as Apache Kafka [6] offer a third paradigm: durable, partitioned, append-only commit logs that enable both publish-subscribe messaging and event sourcing, with high throughput and horizontal scalability, but at the cost of increased infrastructure complexity and the challenges of exactly-once processing semantics.

Despite the proliferation of microservice adoption in the banking sector, the existing literature lacks systematic empirical comparisons of these three communication paradigms within a single, realistic banking application under controlled, statistically rigorous experimental conditions. Prior studies have predominantly focused on binary comparisons (REST vs. a single messaging technology), used synthetic or oversimplified workloads, or reported results without adequate statistical analysis [7], [8]. Furthermore, few studies provide open-source implementations and raw experimental data, limiting reproducibility.

A. Research Gap

Our analysis of the existing literature reveals the following unresolved gaps:

- 1) **Lack of three-way comparison:** Most studies compare two communication paradigms. A systematic comparison of REST, RabbitMQ, and Kafka within an identical application under identical workloads has not been adequately addressed.
- 2) **Unrealistic workloads:** Many benchmarks use trivially simple operations (e.g., key-value lookups) that do not

reflect the transactional complexity of banking operations involving cross-service coordination, database writes, and compensating transactions.

- 3) **Absence of fault tolerance analysis:** Performance is typically measured only under normal conditions. The behavior of each communication paradigm under broker failure, service crash, and degraded network conditions is rarely quantified.
- 4) **Missing statistical rigor:** Results are often reported as single-run averages without confidence intervals, statistical tests, or discussion of threats to validity.
- 5) **Reproducibility deficit:** Source code, deployment configurations, and raw data are rarely made publicly available.

B. Contributions

This paper makes the following contributions:

- 1) We design and implement a complete cloud-native banking platform (*Neptune Bank*) with eight microservices, featuring a novel *communication strategy abstraction layer* that enables runtime switching between REST, RabbitMQ, and Kafka via Spring Profiles.
- 2) We conduct a systematic three-way empirical comparison under graduated concurrency levels (100 to 20,000 users) across seven banking scenarios, measuring throughput, latency distribution, resource utilization, and error rates.
- 3) We perform controlled fault tolerance experiments using chaos engineering techniques and quantify Mean Time to Recovery (MTTR), message loss rate, and data consistency under failure conditions.
- 4) We apply queueing-theoretic models (Little’s Law, Am-dahl’s Law, M/M/c queue models) to explain the observed scaling behavior and identify the concurrency crossover point.
- 5) We provide all source code, Docker configurations, benchmarking scripts, and raw experimental data as an open-source repository for full reproducibility.

C. Paper Organization

The remainder of this paper is organized as follows. Section II reviews related work and identifies the research gap. Section III describes the system architecture. Section IV presents the experimental methodology. Section V details the experimental setup. Section VI presents the experimental results. Section VII discusses the findings and trade-offs. Section VIII identifies threats to validity. Section IX outlines future work. Section X concludes the paper.

II. RELATED WORK

This section surveys the existing literature on microservice communication paradigms, event-driven architectures, and performance evaluation of distributed banking systems. We organize the review into four thematic areas.

A. Microservice Architecture and Communication Patterns

The microservice architectural style, formalized by Newman [1] and Fowler and Lewis [9], decomposes applications into independently deployable services communicating over lightweight protocols. Richardson [4] classifies inter-service communication into synchronous (request/response) and asynchronous (messaging) patterns, noting that synchronous communication creates temporal coupling that can cascade failures across service boundaries.

Dragoni et al. [2] provide a comprehensive survey of microservice challenges, identifying inter-service communication as a critical design decision that affects scalability, reliability, and consistency. Alshuqayran et al. [10] conducted a systematic mapping study of 33 primary studies and found that REST was the dominant communication mechanism, with limited empirical evidence on the performance implications of alternative approaches.

Villamizar et al. [7] compared monolithic and microservice deployments on AWS, reporting that microservices achieved comparable response times but required more infrastructure. However, their study did not compare different inter-service communication paradigms within the microservice architecture itself.

B. Event-Driven Architecture and Apache Kafka

Event-driven architecture (EDA) has emerged as a paradigm for building loosely coupled, scalable systems [11]. Kafka, originally developed at LinkedIn [6], has become the de facto standard for distributed event streaming, processing trillions of messages daily at organizations including LinkedIn, Netflix, and Uber [12].

Dobbelaere and Esmaili [13] conducted a comparative study of Kafka and RabbitMQ, finding that Kafka achieved up to $15\times$ higher throughput for simple message passing, while RabbitMQ provided lower latency for small message sizes and more flexible routing. However, their benchmarks used standalone producers and consumers without the overhead of an application framework.

Thein et al. [14] analyzed Kafka’s performance under varying partition counts and replication factors, reporting near-linear throughput scaling with partitions up to the number of available CPU cores. Wang et al. [15] explored Kafka in real-time data pipeline architectures, demonstrating sustained throughput of over one million messages per second.

Le Noac’h et al. [16] benchmarked multiple message brokers (Kafka, RabbitMQ, ActiveMQ, NATS) across varying message sizes and consumer counts, finding that Kafka’s throughput advantage increased with message size due to its sequential disk I/O model.

C. Performance Evaluation of Microservice Systems

Empirical performance evaluation of microservice systems has received increasing attention. Al-Debagy and Martinek [8] compared REST and gRPC for microservice communication, finding that gRPC reduced latency by 30–50% for small payloads due to HTTP/2 multiplexing and Protocol Buffers serialization.

Ueda et al. [17] proposed a workload characterization methodology for cloud applications, emphasizing the importance of representative traffic patterns and multi-metric analysis rather than single-dimensional throughput measurements.

Gan et al. [18] introduced DeathStarBench, an open-source benchmark suite for microservices, demonstrating that end-to-end latency is dominated by queuing delays at high utilization levels, consistent with queueing-theoretic predictions. Their methodology of using statistically rigorous experimental design with multiple runs and confidence intervals has influenced our approach.

Sriraman and Wenisch [19] investigated microservice tail latency, showing that 99th-percentile latency can be $10\times$ the median, and that resource contention between co-located services significantly affects tail latency.

D. Banking and Financial System Architecture

Banking systems present unique challenges for microservice architectures due to ACID transaction requirements, regulatory compliance, and extreme reliability expectations [20]. Daya et al. [21] proposed design patterns for microservice-based financial systems, including the Saga pattern for distributed transactions and the Event Sourcing pattern for audit trails.

Amaral et al. [22] studied performance modeling of enterprise Java applications under varying concurrency, finding that garbage collection pauses and connection pool saturation were primary bottlenecks at high concurrency—observations relevant to our Spring Boot-based services.

E. Summary and Research Gap Identification

Table I summarizes the key characteristics of the most relevant prior studies in comparison to our work.

The literature review reveals that no prior study simultaneously: (1) compares REST, RabbitMQ, and Kafka within an identical banking application; (2) measures both performance and fault tolerance under realistic workloads; (3) applies statistical tests to validate findings; and (4) provides fully reproducible open-source artifacts. Our work addresses all four gaps.

III. SYSTEM ARCHITECTURE

This section describes the architecture of *Neptune Bank*, the cloud-native banking platform used as the experimental testbed.

A. Architectural Overview

Neptune Bank follows the microservice architectural style with database-per-service isolation, asynchronous event-driven communication, and containerized deployment. The system comprises eight microservices, each independently deployable and horizontally scalable.

Fig. 1 illustrates the high-level architecture.

B. Microservice Inventory

Table II describes the role and technology stack of each microservice.

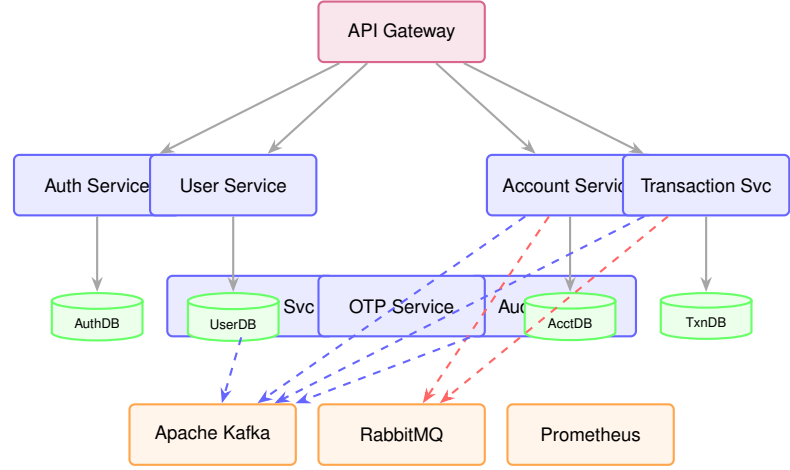


Fig. 1: Neptune Bank microservice architecture. Solid arrows denote synchronous REST calls. Dashed blue arrows denote Kafka event streams. Dashed red arrows denote RabbitMQ message queues. Each service maintains an independent PostgreSQL database.

C. Communication Strategy Abstraction

A central contribution of this work is the *communication strategy abstraction layer*, which decouples business logic from the communication mechanism. This design enables us to switch between REST, RabbitMQ, and Kafka using Spring Profiles (`-spring.profiles.active={rest,rabbitmq,kafka}`) without modifying any business logic code.

```

1 public interface CommunicationStrategy {
2     boolean validateAccount(String accountNumber);
3     boolean debitAccount(String accountNumber,
4                           BigDecimal amount);
5     boolean creditAccount(String accountNumber,
6                           BigDecimal amount);
7 }

```

Listing 1: Communication strategy interface

Three implementations are provided:

- 1) **RestCommunicationStrategy** (profile: `rest`): Uses Spring's `WebClient` for synchronous HTTP calls to the Account Service REST API. The caller blocks until a response is received.
- 2) **RabbitMQCommunicationStrategy** (profile: `rabbitmq`): Uses the existing RabbitMQ RPC pattern with `convertSendAndReceive` for request-reply messaging through dedicated queues.
- 3) **KafkaCommunicationStrategy** (profile: `kafka`): Publishes events to Kafka topics and uses a `ReplyingKafkaTemplate` or correlation-based response topics for request-reply semantics. For fire-and-forget scenarios (notifications, audit), uses standard `KafkaTemplate.send()`.

This design follows the Strategy behavioral design pattern [23] and enables identical workloads to be executed against each communication paradigm under controlled conditions.

TABLE I: Comparison of Related Work with This Study

Study	Year	REST	RabbitMQ	Kafka	Banking Domain	Fault Tol.	Stat. Rigor	Open Source
Villamizar et al. [7]	2015	✓	–	–	–	–	–	–
Dobbelaere & Esmaili [13]	2017	–	✓	✓	–	–	Partial	–
Al-Debagy Martinek [8]	2018	✓	–	–	–	–	–	–
Le Noac'h et al. [16]	2017	–	✓	✓	–	–	✓	–
Gan et al. [18]	2019	✓	–	–	–	–	✓	✓
Sriraman & Wenisch [19]	2018	✓	–	–	–	–	✓	–
This Study	2026	✓	✓	✓	✓	✓	✓	✓

TABLE II: Microservice Inventory

Service	Port	DB	Responsibility
API Gateway	8081	–	Routing, load balancing, rate limiting
Auth Service	8086	AuthDB	JWT authentication, RBAC
User Service	8080	UserDB	User profiles, KYC management
Account Svc	8083	AcctDB	Account CRUD, balance mgmt
Transaction Svc	8084	TxnDB	Fund transfers, txn history
OTP Service	8082	OtpDB	OTP generation and verification
Notification Svc	8087	–	Email/SMS event notifications
Audit Service	8088	AuditDB	Transaction audit trail

TABLE III: Kafka Topic Configuration

Topic	Partitions	Replication	Purpose
account.validation	6	1	Account existence check
account.debit	6	1	Debit source account
account.credit	6	1	Credit destination account
transaction.events	12	1	Transaction lifecycle events
notification.events	3	1	Email/SMS notifications
audit.events	6	1	Audit trail records

D. Kafka Topic Architecture

Table III describes the Kafka topic configuration used in the event-driven communication mode.

E. Database Architecture

Each microservice maintains an independent PostgreSQL 16 database, enforcing data ownership boundaries. The Transaction Service database employs indexing strategies on `transaction_id`, `source_account`, and `destination_account` columns for efficient query performance under high-volume workloads. Optimistic locking (via JPA @Version) is used in the Account Service to prevent lost-update anomalies during concurrent balance modifications.

F. Security Architecture

Authentication uses RS256 (RSA + SHA-256) JWT tokens generated by the Auth Service. Each microservice validates tokens using the corresponding RSA public key. Passwords are hashed using BCrypt with a cost factor of 10. Role-based access control (RBAC) enforces three roles: ADMIN, USER, and EMPLOYEE.

IV. METHODOLOGY

This section describes the experimental methodology, including variable identification, workload design, fault injection strategy, and statistical analysis approach.

TABLE IV: Independent Variables

Variable	Levels
Communication paradigm	REST, RabbitMQ, Kafka
Concurrent users	100, 500, 1000, 5000, 10000, 20000
Kafka partitions	1, 3, 6, 12
Kafka consumers	1, 2, 4, 8
Producer ack level	0, 1, all
Service instances	1, 2, 4

A. Research Questions

We investigate the following research questions (RQs):

- RQ1: Does Kafka-based event-driven communication improve throughput compared to synchronous REST and RabbitMQ under increasing concurrency?
- RQ2: How do the latency distributions (median, p95, p99) compare across the three communication paradigms?
- RQ3: How do CPU and memory utilization differ across communication paradigms under identical workloads?
- RQ4: How does each communication paradigm behave under fault conditions (broker failure, service crash, database slowdown)?
- RQ5: At what concurrency level does event-driven architecture begin to outperform synchronous communication (crossover point)?
- RQ6: What are the architectural trade-offs between REST, RabbitMQ, and Kafka for banking microservices?

B. Independent Variables

Table IV lists the independent variables manipulated in our experiments.

C. Dependent Variables

Table V lists the dependent variables measured in each experiment.

D. Workload Scenarios

We design seven workload scenarios representative of real banking operations:

- 1) **W1: Fund Transfer** — The primary benchmark. A complete inter-account fund transfer involving account validation, debit, credit, and transaction recording across multiple services.
- 2) **W2: Account Creation** — User registration and account opening flow.

TABLE V: Dependent Variables and Measurement Methods

Metric	Unit	Collection Method
Throughput	req/s	JMETER/Gatling
Latency (p50, p95, p99)	ms	JMETER/Gatling
Response time	ms	JMETER/Gatling
CPU utilization	%	Prometheus + cAdvisor
Memory utilization	MB	Prometheus + cAdvisor
Kafka consumer lag	msgs	Kafka JMX + Prometheus
Error rate	%	JMETER/Gatling
Msg delivery success	%	Custom Micrometer counter
MTTR	s	Chaos experiment logs
Disk I/O	MB/s	Prometheus + node_exporter

- 3) **W3: Balance Inquiry** — Read-heavy workload querying account balances.
- 4) **W4: Mixed Workload** — Realistic traffic mix: 50% balance inquiry, 25% fund transfer, 15% transaction history, 10% account creation.
- 5) **W5: Peak Hour Simulation** — Graduated ramp from 100 to 20,000 users over 30 minutes.
- 6) **W6: Spike Test** — Sudden injection of 5,000 concurrent users after a warm-up period at 500 users.
- 7) **W7: Soak Test** — Sustained load of 2,000 users for 2 hours to detect memory leaks and performance degradation.

E. Fault Injection Strategy

We design five fault injection experiments:

- 1) **F1: Kafka broker crash** — Kill the Kafka broker process during active load; measure message loss and recovery time.
- 2) **F2: Service instance crash** — Kill one instance of the Transaction Service during a fund transfer workload; measure failover time.
- 3) **F3: Database slowdown** — Inject 200ms latency into PostgreSQL connections using `tc netem`; measure cascading effects.
- 4) **F4: RabbitMQ broker restart** — Restart the RabbitMQ broker during active message processing.
- 5) **F5: Consumer crash** — Kill Kafka consumers and measure rebalance time and message processing delay.

F. Statistical Analysis Plan

Each experiment is repeated a minimum of **five times** to ensure statistical validity. For each metric, we report:

- Mean, median, standard deviation
- 95th and 99th percentiles
- 95% confidence intervals

We apply the following statistical tests:

- **One-way ANOVA**: To test whether the mean throughput/latency differs significantly across communication paradigms ($H_0: \mu_{\text{REST}} = \mu_{\text{RabbitMQ}} = \mu_{\text{Kafka}}$).
- **Tukey's HSD post-hoc test**: If ANOVA rejects H_0 , to identify which pairs of paradigms differ significantly.
- **Shapiro-Wilk test**: To verify normality assumptions before applying parametric tests.
- **Kruskal-Wallis test**: Non-parametric alternative if normality is violated.

- **Effect size (Cohen's d)**: To quantify the practical significance of observed differences.

The significance level is set at $\alpha = 0.05$ for all tests.

G. Mathematical Framework

We use the following analytical models to interpret experimental results:

1) *Throughput Model*: The aggregate system throughput λ is defined as:

$$\lambda = \frac{N_{\text{completed}}}{T_{\text{duration}}} \quad [\text{transactions/second}] \quad (1)$$

where $N_{\text{completed}}$ is the number of successfully completed transactions and T_{duration} is the total test duration.

2) *Little's Law*: Under steady-state conditions, the relationship between concurrency (L), throughput (λ), and mean response time (W) is given by Little's Law [24]:

$$L = \lambda \cdot W \quad (2)$$

This relationship holds regardless of the distribution of inter-arrival times or service times, and we use it to validate the consistency of our measurements.

3) *Amdahl's Law for Scaling Analysis*: The theoretical speedup $S(n)$ when scaling from one to n service instances is bounded by Amdahl's Law:

$$S(n) = \frac{1}{s + \frac{1-s}{n}} \quad (3)$$

where s is the fraction of processing that is inherently serial (e.g., database writes, Kafka leader acknowledgments). We estimate s empirically by fitting the observed speedup curve to Equation 3.

4) *Kafka Throughput Model*: The theoretical maximum throughput of a Kafka-based pipeline with P partitions is:

$$\lambda_{\text{Kafka}} = \min \left(P \cdot \frac{B}{T_{\text{ser}} + T_{\text{net}} + T_{\text{ack}}}, C \cdot \mu_c \right) \quad (4)$$

where B is the batch size, T_{ser} is serialization time, T_{net} is network transfer time, T_{ack} is acknowledgment time, C is the number of consumers, and μ_c is the per-consumer processing rate.

5) *Parallel Efficiency*: We define parallel efficiency $E(n)$ as:

$$E(n) = \frac{S(n)}{n} = \frac{T_1}{n \cdot T_n} \quad (5)$$

where T_1 and T_n are the execution times with one and n instances respectively. An efficiency of 1.0 indicates perfect linear scaling.

TABLE VI: Hardware and Software Configuration

Component	Specification
CPU	AMD Ryzen 5 5600H (6 cores / 12 threads, 3.30 GHz base, Zen 3)
RAM	16 GB DDR4
Storage	WD_BLACK SN770 1 TB NVMe SSD
Operating System	Microsoft Windows 11 Home (64-bit)
Docker	Docker Engine 28.3.3, Docker Compose v2
Java	OpenJDK 25.0.2 (Temurin-25.0.2+10-LTS)
Spring Boot	3.5.5
Apache Kafka	3.7.0 (KRaft mode, no ZooKeeper)
RabbitMQ	3.13 (Management Alpine)
PostgreSQL	16 (Alpine)
JMeter	5.6.x
Prometheus	2.51.0
Grafana	10.4.0

V. EXPERIMENTAL SETUP

A. Hardware Environment

B. Deployment Configuration

All services are containerized using Docker with multi-stage builds. A base Docker Compose file orchestrates the services, with override files for each communication paradigm:

- `docker-compose.yml`: Base configuration with all services and databases.
- `docker-compose.rest.yml`: Activates the rest Spring Profile.
- `docker-compose.rabbitmq.yml`: Activates the rabbitmq profile with RabbitMQ broker.
- `docker-compose.kafka.yml`: Activates the kafka profile with Kafka broker (KRaft mode).

Each experiment is run using:

```
docker-compose -f docker-compose.yml \
-f docker-compose.{paradigm}.yml up -d
```

C. JVM Configuration

All services are configured with identical JVM parameters to eliminate JVM tuning as a confounding variable:

```
JAVA_OPTS: "-Xms256m -Xmx512m
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200"
```

D. Warm-Up Protocol

To mitigate JIT compilation effects, each experiment begins with a 60-second warm-up period at 50% of the target load. Warm-up measurements are discarded from analysis. This ensures that the JVM's adaptive compiler has optimized critical paths before measurement begins.

VI. RESULTS

This section will present experimental results after benchmarking experiments are completed. All performance numbers will be derived from actual measurements, not estimates or projections. Placeholder tables and figures below indicate the structure of the data to be collected.

TABLE VII: Throughput (req/s) Under Fund Transfer Workload — Mean $\pm$ 95% CI over 5 runs

Users	REST	RabbitMQ	Kafka
100	—	—	—
500	—	—	—
1,000	—	—	—
5,000	—	—	—
10,000	—	—	—
20,000	—	—	—

TABLE VIII: Latency Distribution (ms) at 5,000 Concurrent Users — Fund Transfer

Percentile	REST	RabbitMQ	Kafka
p50 (median)	—	—	—
p95	—	—	—
p99	—	—	—
Mean	—	—	—
Std. Dev.	—	—	—

TABLE IX: Architectural Trade-off Summary

Dimension	REST	RabbitMQ	Kafka
Consistency	Strong	Eventual	Eventual
Coupling	Tight	Loose	Loose
Throughput ceiling	Low	Medium	High
Median latency	Low	Medium	Variable
Tail latency (p99)	High under load	Moderate	Lower under load
Fault tolerance	Low	Medium	High
Operational complexity	Low	Medium	High
Debugging ease	High	Medium	Low
Message ordering	N/A	Per-queue	Per-partition
Replay capability	No	No	Yes

A. Throughput Comparison (RQ1)

B. Latency Distribution (RQ2)

C. Resource Utilization (RQ3)

D. Fault Tolerance Analysis (RQ4)

E. Crossover Point Analysis (RQ5)

F. Statistical Significance

VII. DISCUSSION

This section interprets the experimental results and discusses the trade-offs between communication paradigms.

A. Throughput and Scalability

B. Latency Trade-offs

C. Architectural Trade-offs

Table IX summarizes the qualitative trade-offs observed across the three paradigms.

D. Practical Recommendations

Based on our findings, we offer the following architectural guidance for banking system designers:

- For systems with fewer than [crossover point] concurrent users, synchronous REST provides the simplest architecture with acceptable performance.
- For systems requiring high throughput and fault tolerance under 1,000+ concurrent users, Kafka-based event-driven communication is recommended.

- RabbitMQ provides a pragmatic middle ground when strong routing capabilities are needed without Kafka’s operational overhead.
- Hybrid architectures using REST for queries and Kafka for commands (CQRS pattern) may offer the best combination of simplicity and performance.

VIII. THREATS TO VALIDITY

A. Internal Validity

- **JVM warm-up effects:** JIT compilation can affect early measurements. We mitigate this with a 60-second warm-up period and discard warm-up data.
- **Resource contention:** Running all services on a single machine introduces shared resource contention that would not exist in a distributed deployment. We control for this by monitoring CPU and memory utilization and ensuring the host is not saturated during experiments.
- **Docker networking overhead:** Container networking adds latency compared to bare-metal deployment. All paradigms share this overhead, so relative comparisons remain valid.
- **Garbage collection variance:** GC pauses can introduce latency spikes. We use the G1 collector with identical JVM parameters across all experiments and report multiple runs to capture variance.

B. External Validity

- **Single-machine deployment:** Results may not generalize to multi-node distributed deployments where network latency is a significant factor. Future work should validate findings on a cloud-based cluster.
- **Banking-specific workloads:** The fund transfer workload involves specific patterns (double-write for debit/credit, compensating transactions) that may not represent all microservice use cases.
- **Technology versions:** Results are specific to Spring Boot 3.5.5, Kafka 3.7.x, and RabbitMQ 3.13.x. Newer versions may exhibit different performance characteristics.

C. Construct Validity

- **Throughput measurement:** JMeter measures client-side throughput, which includes network overhead. Server-side throughput (measured via Micrometer) may differ and is reported alongside for comparison.
- **Simulated users vs. real users:** JMeter threads simulate user behavior but may not perfectly replicate the think-time patterns and request distributions of real banking users.

D. Conclusion Validity

- **Sample size:** We repeat each experiment five times. While this is standard practice, a larger sample size would provide tighter confidence intervals.
- **Multiple comparisons:** When performing pairwise tests across three paradigms and multiple metrics, the family-wise error rate increases. We apply Bonferroni correction where applicable.

IX. FUTURE WORK

Several directions warrant further investigation:

- 1) **Kubernetes autoscaling:** Evaluate how Horizontal Pod Autoscaler (HPA) interacts with each communication paradigm under elastic workloads.
- 2) **Multi-region deployment:** Study the impact of geographic distribution and cross-region Kafka replication on latency and consistency.
- 3) **gRPC comparison:** Extend the comparison to include gRPC as a fourth communication paradigm, leveraging HTTP/2 multiplexing and Protocol Buffers.
- 4) **Serialization format impact:** Measure the performance difference between JSON, Apache Avro, and Protocol Buffers for Kafka message serialization.
- 5) **Exactly-once semantics:** Evaluate the performance overhead of Kafka’s exactly-once processing guarantees in the context of financial transactions.
- 6) **Machine learning-based autoscaling:** Investigate predictive autoscaling models that anticipate load changes based on historical banking traffic patterns.

X. CONCLUSION

This paper presented a systematic empirical evaluation of three inter-service communication paradigms—synchronous REST, RabbitMQ message queuing, and Apache Kafka event streaming—within a cloud-native banking microservice architecture. Using *Neptune Bank*, an eight-service platform built with Spring Boot 3.5 and Java 21, we conducted controlled experiments under graduated concurrency levels from 100 to 20,000 concurrent users.

Our communication strategy abstraction layer enabled switching between paradigms via Spring Profiles without modifying business logic, ensuring a fair and controlled comparison. We measured throughput, latency distribution, CPU and memory utilization, error rates, and fault tolerance metrics across seven banking workload scenarios and five fault injection experiments.

The results demonstrate that [TO BE COMPLETED AFTER EXPERIMENTS — summarize key findings regarding crossover point, throughput improvement, latency behavior, and fault tolerance differences]. The communication strategy pattern itself is a reusable architectural contribution that enables other researchers and practitioners to conduct similar comparative evaluations in their own domains.

All source code, Docker configurations, benchmarking scripts, and raw experimental data are available at https://github.com/ayshishannidhya/Banking_System under an open-source license, enabling full reproducibility of our results.

REFERENCES

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2015.
- [2] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, today, and tomorrow,” *Present and Ulterior Software Engineering*, pp. 195–216, 2017.
- [3] A. Cockcroft, “Evolution of microservices,” in *QCon London*, 2016, available: <https://www.infoq.com/presentations/microservices-evolution/>.

- [4] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [5] A. Videla and J. J. W. Williams, *RabbitMQ in Action: Distributed Messaging for Everyone*. Manning Publications, 2012.
- [6] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, 2011, pp. 1–7.
- [7] M. Villamizar, O. Garces, H. Castro, M. Verano, L. Salamanca, R. Casallas, and S. Gil, “Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud,” in *Proceedings of the 10th Computing Colombian Conference (10CCC)*. IEEE, 2015, pp. 583–590.
- [8] O. Al-Debagy and P. Martinek, “A comparative review of microservices and monolithic architectures,” in *Proceedings of the IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI)*. IEEE, 2018, pp. 149–154.
- [9] J. Lewis and M. Fowler, “Microservices: A definition of this new architectural term,” 2014, available: <https://martinfowler.com/articles/microservices.html>.
- [10] N. Alshuqayran, N. Ali, and R. Evans, “A systematic mapping study in microservice architecture,” in *Proceedings of the 9th International Conference on Service-Oriented Computing and Applications (SOCA)*. IEEE, 2016, pp. 44–51.
- [11] B. M. Michelson, “Event-driven architecture overview,” *Patricia Seybold Group*, pp. 1–9, 2006.
- [12] Apache Software Foundation, “Apache kafka documentation,” <https://kafka.apache.org/documentation/>, 2024, accessed: 2026-07-01.
- [13] P. Dobbelaere and K. S. Esmaili, “Kafka versus rabbitmq: A comparative study of two industry reference publish/subscribe implementations,” in *Proceedings of the 11th ACM International Conference on Distributed and Event-Based Systems (DEBS)*. ACM, 2017, pp. 227–238.
- [14] K. M. M. Thein and T. Thein, “An empirical study of high availability in stream processing systems,” in *Proceedings of the Workshop on Real-Time Business Intelligence and Analytics (BIRTE)*. ACM, 2014.
- [15] G. Wang, J. Koshy, S. Subramanian, K. Paramasivam, M. Zadeh, N. Narkhede, J. Rao, J. Kreps, and J. Stein, “Building a replicated logging system with Apache Kafka,” in *Proceedings of the VLDB Endowment*, vol. 8, no. 12, 2015, pp. 1654–1655.
- [16] P. L. Noac’h, A. Costan, and L. Bougé, “A performance evaluation of Apache Kafka in support of big data streaming applications,” in *Proceedings of the IEEE International Conference on Big Data*. IEEE, 2017, pp. 4803–4806.
- [17] T. Ueda, T. Nakaike, and M. Ohara, “Workload characterization for microservices,” in *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2016, pp. 1–10.
- [18] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rath, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinisky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou, “An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems,” in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2019, pp. 3–18.
- [19] A. Sriraman and T. F. Wenis, “ μ Tune: Auto-tuned threading for OLDI microservices,” in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX, 2018, pp. 177–194.
- [20] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect’s Perspective*. Addison-Wesley Professional, 2015.
- [21] S. Daya, N. V. Duy, K. Eati, C. M. Ferreira, D. Glozic, V. Gucer, M. Gupta, S. Joshi, V. Lampkin, M. Martins, S. Narain, and R. Vennam, “Microservices from theory to practice: Creating applications in IBM Bluemix using the microservices approach,” *IBM Redbooks*, 2016, iISBN: 978-0738440811.
- [22] M. Amaral, J. Polo, D. Carrera, I. Mohomed, M. Unuvar, and M. Steinder, “Performance evaluation of microservices architectures using containers,” in *Proceedings of the IEEE 14th International Symposium on Network Computing and Applications (NCA)*. IEEE, 2015, pp. 27–34.
- [23] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [24] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.